

AE 08 - The Office - Predicting IMDB Ratings

Max Asmar

2026-05-14

Contents

Introduction	1
Getting Started	2
Packages	2
The Data	2
Our Goal	3
Exercise 1: Calculate Line Percentages	3
Exercise 2: Identify Special Episodes	5
Exercise 3: Create Modeling Dataset	6
Exercise 4: Split the Data	8
Exercise 5: Specify a Model	8
Exercise 6: Create a Recipe	9
Exercise 7: Build a Workflow	9
Exercise 8: Fit the Model	10
Exercise 9: Cross-Validation	11
Exercise 10: Compare Models	12
Exercise 11: Challenge (Optional)	14
Reflection	14
Knit and Submit	15
Generate PDF for Canvas	15
Troubleshooting	15
Key Concepts Review	16
Quick Reference	16

Introduction

In this application exercise, you'll build a predictive model for IMDB ratings of episodes of *The Office* using the **tidymodels** framework. You'll practice feature engineering, creating modeling workflows, and evaluating model performance.

In this application exercise, you will:

- Engineer features from text data (episode scripts)
- Create indicator variables for special episodes
- Build a modeling dataset
- Split data into training and testing sets
- Specify and fit linear regression models
- Create preprocessing recipes
- Build modeling workflows
- Perform cross-validation
- Compare model performance

Estimated time: 60-90 minutes

Getting Started

Fork and Clone Workflow

Quick reminder: Work in YOUR forked `application-exercises` repository.

1. Navigate to the `ae-08-office` folder in YOUR fork
 2. Open `ae-08-office.Rmd`
 3. Update the YAML with your name
 4. Knit to verify it works
-

Packages

```
library(tidyverse)
library(tidymodels)
library(schrute)      # Contains The Office data
library(lubridate)    # For working with dates
```

What these packages do:

- **tidyverse:** Data wrangling and visualization
 - **tidymodels:** Modeling framework (replaces caret)
 - **schrute:** Contains transcripts from The Office
 - **lubridate:** Working with dates
-

The Data

Use `theoffice` data from the **schrute** package to predict IMDB scores for episodes of The Office.

```
glimpse(theoffice)

## Rows: 55,130
## Columns: 12
## $ index      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16~
## $ season     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ episode    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ episode_name <chr> "Pilot", "Pilot", "Pilot", "Pilot", "Pilot", "Pilot",~
## $ director   <chr> "Ken Kwapis", "Ken Kwapis", "Ken Kwapis", "Ken Kwapis~
## $ writer     <chr> "Ricky Gervais;Stephen Merchant;Greg Daniels", "Ricky~
## $ character  <chr> "Michael", "Jim", "Michael", "Jim", "Michael", "Micha~
## $ text       <chr> "All right Jim. Your quarterlies look very good. How ~
## $ text_w_direction <chr> "All right Jim. Your quarterlies look very good. How ~
## $ imdb_rating <dbl> 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6, 7.6,~
## $ total_votes <int> 3706, 3706, 3706, 3706, 3706, 3706, 3706, 3706, 3706,~
## $ air_date   <chr> "2005-03-24", "2005-03-24", "2005-03-24", "2005-03-24~
```

Understanding the data:

- Each row is a line of dialogue
- Multiple rows per episode (one per line)
- Variables include: `season`, `episode`, `character`, `text`, `imdb_rating`, `total_votes`, `air_date`

Fix `air_date` for later use:

```
theoffice <- theoffice %>%
  mutate(air_date = ymd(as.character(air_date)))
```

What this does:

- Converts `air_date` to proper Date format
 - `ymd()`: Parses “year-month-day” format
-

Our Goal

We will:

- Engineer features based on episode scripts
- Train a model
- Perform cross validation
- Make predictions

Note: The episodes listed in `theoffice` don’t match the ones listed in the data we used in the cross validation lesson.

```
theoffice %>%
  distinct(season, episode)
```

```
## # A tibble: 186 x 2
##   season episode
##   <int>   <int>
## 1      1       1
## 2      1       2
## 3      1       3
## 4      1       4
## 5      1       5
## 6      1       6
## 7      2       1
## 8      2       2
## 9      2       3
## 10     2       4
## # i 176 more rows
```

Exercise 1: Calculate Line Percentages

Calculate the percentage of lines spoken by Jim, Pam, Michael, and Dwight for each episode of The Office.

Why? These are main characters, and the amount they speak might relate to episode ratings.

Task 1.1: Create a dataset with line percentages.

```
# Remember to change eval=FALSE to eval=TRUE!

office_lines <- theoffice %>%
  # Group by season and episode
  group_by(season, episode) %>%
  # Calculate total lines per episode and lines by main characters
  summarise(
    total_lines = n(),
    jim_lines = sum(character == "Jim"),
```

```

pam_lines = sum(character == "Pam"),
michael_lines = sum(character == "Michael"),
dwight_lines = sum(character == "Dwight")
) %>%
# Calculate percentages
mutate(
  jim_pct = jim_lines / total_lines,
  pam_pct = pam_lines / total_lines,
  michael_pct = michael_lines / total_lines,
  dwight_pct = dwight_lines / total_lines
)

office_lines

## # A tibble: 186 x 11
## # Groups:   season [9]
##   season episode total_lines jim_lines pam_lines michael_lines dwight_lines
##   <int>   <int>     <int>    <int>    <int>         <int>         <int>
## 1     1     1         229      36      41           81           29
## 2     1     2         203      25      12           75           17
## 3     1     3         244      42      32           56           62
## 4     1     4         243      49      22           68           47
## 5     1     5         230      21      14          104           25
## 6     1     6         346      55      45          106           28
## 7     2     1         256      32      41           96           32
## 8     2     2         283      16      27          100           11
## 9     2     3         281      55      33           83           55
## 10    2     4         319      51      22           69           65
## # i 176 more rows
## # i 4 more variables: jim_pct <dbl>, pam_pct <dbl>, michael_pct <dbl>,
## #   dwight_pct <dbl>

```

Hints:

- Group by season and episode
- Character names: "Jim", "Pam", "Michael", "Dwight"
- `sum(character == "Jim")` counts lines spoken by Jim
- `n()` gives total number of rows (lines) in each group

Task 1.2: Which episode has the highest percentage of Michael lines?

Remember to change eval=FALSE to eval=TRUE!

```

office_lines %>%
  arrange(desc(michael_pct)) %>%
  select(season, episode, michael_pct)

```

```

## # A tibble: 186 x 3
## # Groups:   season [9]
##   season episode michael_pct
##   <int>   <int>         <dbl>
## 1     1     5         0.452
## 2     2     5         0.401
## 3     2    14         0.387
## 4     7    17         0.387
## 5     2     8         0.377

```

```
## 6      2      1      0.375
## 7      3     23      0.374
## 8      1      2      0.369
## 9      5     19      0.363
## 10     3      1      0.362
## # i 176 more rows
```

Your answer: Season 1, Episode 5

Exercise 2: Identify Special Episodes

Identify episodes that touch on Halloween, Valentine's Day, and Christmas.

Why? Special holiday episodes might have different ratings patterns.

Task 2.1: Search episode text for holiday keywords.

Remember to change eval=FALSE to eval=TRUE!

```
office_episodes <- theoffice %>%
  group_by(season, episode) %>%
  summarise(
    # Check if text contains holiday keywords (case-insensitive)
    halloween = sum(str_detect(str_to_lower(text), "halloween")) > 0,
    valentine = sum(str_detect(str_to_lower(text), "valentine")) > 0,
    christmas = sum(str_detect(str_to_lower(text), "christmas")) > 0
  ) %>%
  # Convert TRUE/FALSE to 1/0
  mutate(
    halloween = as.numeric(halloween),
    valentine = as.numeric(valentine),
    christmas = as.numeric(christmas)
  )

office_episodes
```

```
## # A tibble: 186 x 5
## # Groups:   season [9]
##   season episode halloween valentine christmas
##   <int>   <int>     <dbl>     <dbl>     <dbl>
## 1     1       1       0         0         0
## 2     1       2       0         0         0
## 3     1       3       0         0         0
## 4     1       4       0         0         0
## 5     1       5       0         0         0
## 6     1       6       0         0         0
## 7     2       1       0         0         0
## 8     2       2       0         0         0
## 9     2       3       0         0         0
## 10    2       4       0         0         0
## # i 176 more rows
```

Hints:

- Search for keywords: “halloween”, “valentine”, “christmas”
- `str_detect()`: Returns TRUE if pattern found

- `str_to_lower()`: Converts text to lowercase
- `sum(...)` > 0: TRUE if found at least once
- `as.numeric(TRUE)` = 1, `as.numeric(FALSE)` = 0

Task 2.2: How many episodes mention each holiday?

Remember to change eval=FALSE to eval=TRUE!

```
office_episodes %>%
  ungroup() %>%
  summarise(
    halloween_episodes = sum(halloween),
    valentine_episodes = sum(valentine),
    christmas_episodes = sum(christmas),
  )

## # A tibble: 1 x 3
##   halloween_episodes valentine_episodes christmas_episodes
##           <dbl>           <dbl>           <dbl>
## 1             10             7             34
```

Your answers:

Halloween episodes: 10 episodes Valentine episodes: 7 episodes Christmas episodes: 34 episodes —

Exercise 3: Create Modeling Dataset

Put together a modeling dataset that includes features you've engineered. Also add an indicator variable called `michael` which takes the value 1 if Michael Scott (Steve Carrell) was there, and 0 if not.

Note: Michael Scott left the show at the end of Season 7.

Task 3.1: Get one row per episode with all features.

Remember to change eval=FALSE to eval=TRUE!

```
# First, get the basic episode info (one row per episode)
office_info <- theoffice %>%
  group_by(season, episode) %>%
  summarise(
    episode_name = first(episode_name),
    imdb_rating = first(imdb_rating),
    total_votes = first(total_votes),
    air_date = first(air_date)
  )

# Join everything together
office_df <- office_info %>%
  # Join with line percentages
  left_join(office_lines, by = c("season", "episode")) %>%
  # Join with holiday indicators
  left_join(office_episodes, by = c("season", "episode")) %>%
  # Add Michael indicator (1 if season <= 7, 0 otherwise)
  mutate(michael = if_else(season <= 7, 1, 0))

office_df

## # A tibble: 186 x 19
```

```
## # Groups:   season [9]
##   season episode episode_name   imdb_rating total_votes air_date   total_lines
##   <int>   <int> <chr>             <dbl>       <int> <date>         <int>
## 1     1     1     1 Pilot                7.6         3706 2005-03-24       229
## 2     1     2     2 Diversity Day          8.3         3566 2005-03-29       203
## 3     1     3     3 Health Care           7.9         2983 2005-04-05       244
## 4     1     4     4 The Alliance           8.1         2886 2005-04-12       243
## 5     1     5     5 Basketball           8.4         3179 2005-04-19       230
## 6     1     6     6 Hot Girl             7.8         2852 2005-04-26       346
## 7     2     1     1 The Dundies           8.7         3213 2005-09-20       256
## 8     2     2     2 Sexual Harassm~       8.2         2736 2005-09-27       283
## 9     2     3     3 Office Olympics       8.4         2742 2005-10-04       281
## 10    2     4     4 The Fire              8.4         2713 2005-10-11       319
## # i 176 more rows
## # i 12 more variables: jim_lines <int>, pam_lines <int>, michael_lines <int>,
## #   dwight_lines <int>, jim_pct <dbl>, pam_pct <dbl>, michael_pct <dbl>,
## #   dwight_pct <dbl>, halloween <dbl>, valentine <dbl>, christmas <dbl>,
## #   michael <dbl>
```

Hints:

- Join with office_lines
- Join with office_episodes
- Join by "season" and "episode"
- Michael left after season 7: season <= 7

Task 3.2: Verify the dataset.

```
# Remember to change eval=FALSE to eval=TRUE!
```

```
# Check dimensions
```

```
dim(office_df)
```

```
## [1] 186 19
```

```
# Check for missing values
```

```
office_df %>%
  summarise(across(everything(), ~sum(is.na(.))))
```

```
## # A tibble: 9 x 19
```

```
##   season episode episode_name imdb_rating total_votes air_date total_lines
##   <int>   <int>   <int>         <int>       <int>   <int>         <int>
## 1     1     0     0             0           0       0           0
## 2     2     0     0             0           0       0           0
## 3     3     0     0             0           0       0           0
## 4     4     0     0             0           0       0           0
## 5     5     0     0             0           0       0           0
## 6     6     0     0             0           0       0           0
## 7     7     0     0             0           0       0           0
## 8     8     0     0             0           0       0           0
## 9     9     0     0             0           0       0           0
```

```
## # i 12 more variables: jim_lines <int>, pam_lines <int>, michael_lines <int>,
## #   dwight_lines <int>, jim_pct <int>, pam_pct <int>, michael_pct <int>,
## #   dwight_pct <int>, halloween <int>, valentine <int>, christmas <int>,
## #   michael <int>
```

```
# Check Michael variable
```

```
office_df %>%
  count(season, michael)
```

```
## # A tibble: 9 x 3
## # Groups:   season [9]
##   season michael     n
##   <int>    <dbl> <int>
## 1     1         1     6
## 2     2         1    22
## 3     3         1    23
## 4     4         1    14
## 5     5         1    26
## 6     6         1    24
## 7     7         1    24
## 8     8         0    24
## 9     9         0    23
```

How many episodes are in the dataset? There are 186 episodes in the dataset.

Do seasons 1-7 have michael = 1? Yes.

Exercise 4: Split the Data

Split the data into training (75%) and testing (25%).

```
# Remember to change eval=FALSE to eval=TRUE!

set.seed(1122)

office_split <- initial_split(office_df, prop = 0.75)
office_train <- training(office_split)
office_test  <- testing(office_split)

# Check the split
nrow(office_train)

## [1] 139
nrow(office_test)
```

```
## [1] 47
```

Hints:

- `initial_split(office_df, prop = 0.75)`
- `training(office_split)`
- `testing(office_split)`

Training set size: 139 episodes

Testing set size: 47 episodes

Exercise 5: Specify a Model

Specify a linear regression model.


```
# Remember to change eval=FALSE to eval=TRUE!
```

```
office_mod <- linear_reg() %>%  
  set_engine("lm")
```

Hint: Use "lm" as the engine for linear regression.

Exercise 6: Create a Recipe

Create a recipe that:

- Updates the role of `episode_name` to not be a predictor
- Removes `air_date` as a predictor
- Uses `season` as a factor
- Removes all zero variance predictors

```
# Remember to change eval=FALSE to eval=TRUE!
```

```
office_rec <- recipe(imdb_rating ~ ., data = office_train) %>%  
  # Update role of episode_name (keep it but don't use as predictor)  
  update_role(episode_name, new_role = "ID") %>%  
  # Remove air_date  
  step_rm(air_date) %>%  
  # Convert season to factor  
  step_mutate(season = as.factor(season)) %>%  
  # Remove zero variance predictors  
  step_zv(all_predictors())  
  
# View the recipe  
office_rec
```

Hints:

- Formula: `imdb_rating ~ .` (predict rating with all variables)
 - Data: `office_train`
 - Update role of `episode_name` to "ID"
 - Remove `air_date`
 - Convert `season` to factor
 - Remove zero variance from `all_predictors()`
-

Exercise 7: Build a Workflow

Build a workflow for fitting the model specified earlier and using the recipe you developed to preprocess the data.

```
# Remember to change eval=FALSE to eval=TRUE!
```

```
office_wflow <- workflow() %>%  
  add_model(office_mod) %>%  
  add_recipe(office_rec)
```

```
office_wflow
```

```
## == Workflow =====
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor -----
## 3 Recipe Steps
##
## * step_rm()
## * step_mutate()
## * step_zv()
##
## -- Model -----
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

Hints:

- Add the model: `office_mod`
- Add the recipe: `office_rec`

Exercise 8: Fit the Model

Fit the model to training data and interpret a couple of the slope coefficients.

Remember to change `eval=FALSE` to `eval=TRUE`!

```
office_fit <- office_wflow %>%
  fit(data = office_train)

# View the coefficients
tidy(office_fit) %>% print(n=24)
```

```
## # A tibble: 24 x 5
##   term          estimate std.error statistic    p.value
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 (Intercept)    6.35      0.648      9.81 6.77e-17
## 2 season2        0.473     0.154      3.07 2.67e- 3
## 3 season3        0.708     0.163      4.33 3.16e- 5
## 4 season4        0.728     0.187      3.89 1.66e- 4
## 5 season5        0.655     0.168      3.89 1.68e- 4
## 6 season6        0.396     0.185      2.14 3.42e- 2
## 7 season7        0.608     0.180      3.38 9.73e- 4
## 8 season8        0.0915    0.240      0.381 7.04e- 1
## 9 season9        0.426     0.233      1.83 6.98e- 2
## 10 episode       0.00954    0.00417      2.29 2.38e- 2
## 11 total_votes    0.000313    0.0000410      7.63 7.04e-12
## 12 total_lines    0.00145    0.00196      0.738 4.62e- 1
## 13 jim_lines     -0.000951    0.00823     -0.116 9.08e- 1
## 14 pam_lines      0.00744    0.00841      0.885 3.78e- 1
## 15 michael_lines -0.00483    0.00373     -1.30 1.98e- 1
## 16 dwight_lines   0.000363    0.00882      0.0412 9.67e- 1
## 17 jim_pct        0.862      2.48      0.348 7.28e- 1
## 18 pam_pct       -2.72      2.62     -1.04 3.02e- 1
```

```
## 19 michael_pct      1.78      1.17      1.52      1.31e- 1
## 20 dwight_pct       0.408     2.44      0.167     8.68e- 1
## 21 halloween        0.0394    0.124     0.319     7.50e- 1
## 22 valentine        0.0290    0.141     0.205     8.38e- 1
## 23 christmas        0.154     0.0705    2.19      3.07e- 2
## 24 michael          NA        NA        NA        NA
```

Hint: Fit to office_train

Task 8.1: What is the coefficient for michael?

Your answer: NA

Interpretation:

The michael variable is multicollinear with other variables in the model, such as season, meaning that it provides no new information to the model. Whether or not Michael is in the episode is irrelevant to the IMDB rating. The IMDB rating should be a function of the other variables because the season variable already accounts for Michael's presence. Michael is in season 1 through 7, but not 8 and 9.

Task 8.2: What is the coefficient for jim_pct?

Your answer: 0.862

Interpretation:

For every 1 unit increase in the proportion of lines spoken by Jim, the IMDB rating increases by 0.862 points, on average, holding all else constant.

Exercise 9: Cross-Validation

Perform 5-fold cross validation and view model performance metrics.

Remember to change eval=FALSE to eval=TRUE!

Create folds

```
set.seed(345)
folds <- vfold_cv(office_train, v = 5)
folds
```

```
## # 5-fold cross-validation
## # A tibble: 5 x 2
##   splits      id
##   <list>    <chr>
## 1 <split [111/28]> Fold1
## 2 <split [111/28]> Fold2
## 3 <split [111/28]> Fold3
## 4 <split [111/28]> Fold4
## 5 <split [112/27]> Fold5
```

Fit resamples

```
set.seed(456)
office_fit_rs <- office_wflow %>%
  fit_resamples(folds)
```

Collect metrics

```
collect_metrics(office_fit_rs)
```

```
## # A tibble: 2 x 6
```

```
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 rmse    standard    0.357     5  0.0653 Preprocessor1_Model1
## 2 rsq     standard    0.597     5  0.0263 Preprocessor1_Model1
```

Hints:

- Create folds from `office_train`
- Use `v = 5` for 5-fold CV
- Fit `office_wflow` to folds
- Collect metrics from `office_fit_rs`

Task 9.1: What is the mean RMSE from cross-validation?

Your answer: 0.357

Task 9.2: What is the mean R-squared from cross-validation?

Your answer: 0.597

Task 9.3: What do these metrics tell you about model performance?

Your answer:

The RMSE value tells me that the predicted IMDB ratings from the model were typically 0.357 stars off from the actual IMDB rating.

The R^2 value means that around 59.7% of the variability in IMDB ratings is explained by the season, total votes, total lines, jim lines, pam lines, michael lines, dwight lines, jim pct, pam pct, michael pct, dwight pct, halloween, valentine, and christmas variables.

Exercise 10: Compare Models

Use your model to make predictions for the testing data and calculate the RMSE. Also use the model developed in the cross validation lesson to make predictions for the testing data and calculate the RMSE as well. Which model did a better job in predicting IMDB scores for the testing data?

New Model

```
# Remember to change eval=FALSE to eval=TRUE!

# Make predictions on test data
office_pred <- predict(office_fit, office_test) %>%
  bind_cols(office_test)

# Calculate RMSE
office_pred %>%
  rmse(truth = imdb_rating, estimate = .pred)

## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 rmse    standard      0.400
```

Hints:

- Predict using `office_fit` on `office_test`
- Bind predictions to `office_test`
- RMSE: `truth = imdb_rating, estimate = .pred`

New model RMSE: 0.400

Old Model

```
# Remember to change eval=FALSE to eval=TRUE!

# Specify old model
office_mod_old <- linear_reg() %>%
  set_engine("lm")

# Create old recipe (simpler features)
office_rec_old <- recipe(imdb_rating ~ season + episode + total_votes + air_date,
  data = office_train) %>%
  # Extract month of air_date
  step_date(air_date, features = "month") %>%
  step_rm(air_date) %>%
  # Make dummy variables of month
  step_dummy(contains("month")) %>%
  # Remove zero variance predictors
  step_zv(all_predictors())

# Create old workflow
office_wflow_old <- workflow() %>%
  add_model(office_mod_old) %>%
  add_recipe(office_rec_old)

# Fit old model
office_fit_old <- office_wflow_old %>%
  fit(data = office_train)

# View coefficients
tidy(office_fit_old)

## # A tibble: 12 x 5
##   term                estimate std.error statistic  p.value
##   <chr>                <dbl>    <dbl>    <dbl>    <dbl>
## 1 (Intercept)          7.20      0.188     38.4 9.92e-72
## 2 season             -0.0501    0.0140     -3.57 5.04e- 4
## 3 episode              0.0449    0.00877      5.11 1.13e- 6
## 4 total_votes          0.000360 0.0000404      8.89 4.99e-15
## 5 air_date_month_Feb  -0.145     0.139     -1.04 2.99e- 1
## 6 air_date_month_Mar  -0.376     0.134     -2.81 5.69e- 3
## 7 air_date_month_Apr  -0.309     0.131     -2.36 1.96e- 2
## 8 air_date_month_May  -0.128     0.162     -0.791 4.30e- 1
## 9 air_date_month_Sep   0.512     0.178      2.88 4.63e- 3
## 10 air_date_month_Oct  0.270     0.139      1.95 5.38e- 2
## 11 air_date_month_Nov  0.116     0.126      0.924 3.57e- 1
## 12 air_date_month_Dec  0.407     0.165      2.47 1.49e- 2

# Make predictions with old model
office_pred_old <- predict(office_fit_old, office_test) %>%
  bind_cols(office_test)

# Calculate RMSE for old model
office_pred_old %>%
```

```
rmse(truth = imdb_rating, estimate = .pred)
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 rmse    standard      0.403
```

Old model RMSE: 0.403

Task 10.1: Which model has better performance on the test set?

Your answer:

The new model has better performance because it has a lower RMSE value of 0.400, compared to the old model which has an RMSE value of 0.403.

Task 10.2: Why do you think one model performed better than the other?

Your answer:

The new model typically predicts the IMDB ratings 0.4 stars away from their actual value, compared to the old model which typically predicts the ratings 0.403 stars away from their actual value, given the RMSE values. This means that the new model typically has less error than the old model.

The new model also includes features such as michael_pct and pam_pct that reflect character popularity and acting ability, which could contribute to a better or worse rating.

Exercise 11: Challenge (Optional)

Task 11.1: Create a visualization comparing predicted vs. actual ratings for the test set.

```
# Your code here
office_pred %>%
  ggplot(aes(x = ___, y = ___)) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  labs(
    title = "Predicted vs. Actual IMDB Ratings",
    x = "Actual Rating",
    y = "Predicted Rating"
  )
```

Task 11.2: Which episodes did the model predict least accurately?

```
office_pred %>%
  mutate(error = abs(imdb_rating - .pred)) %>%
  arrange(desc(error)) %>%
  select(season, episode, episode_name, imdb_rating, .pred, error)
```

Reflection

What was the most challenging part of building this model?

I think the most challenging part of building the model was creating a recipe and arranging the variables for the ML model.

What did you learn about feature engineering?

I learned that feature engineering requires a thorough understanding of the library and data in context. I had to use the theoffice help page a few times to understand the variables, before feature engineering them.

If you could add one more feature to improve the model, what would it be?

I would add episode time/length to the model.

Knit and Submit

Before you finish:

1. Make sure all code chunks run without errors
2. Change all `eval=FALSE` to `eval=TRUE`
3. Fill in all answer spaces
4. Knit to HTML first to verify

Git workflow:

1. Click the **Git** pane
 2. Check boxes next to all changed files
 3. Click **Commit**
 4. Write message: “Completed AE 08 - The Office”
 5. Click **Commit** then **Push**
-

Generate PDF for Canvas

1. Click **Knit** dropdown arrow
 2. Select **Knit to PDF**
 3. Find `ae-08-office.pdf` in Files pane
 4. Export and save to your computer
 5. Submit to Canvas
-

Troubleshooting

Common issues:

- **Error: “object not found”:** Make sure you created the object in a previous chunk
- **Error: “could not find function”:** Make sure you loaded all packages
- **Join errors:** Check that you’re joining by the correct column names
- **Cross-validation taking a long time:** This is normal - CV fits multiple models

Data wrangling issues:

- **Line counts seem wrong:** Make sure you grouped by season AND episode
- **Missing values:** Use `na.rm = TRUE` in summary functions
- **Holiday indicators not working:** Make sure you converted to lowercase with `str_to_lower()`

Modeling issues:

- **Recipe errors:** Make sure you’re using `office_train` in the recipe
- **Workflow errors:** Make sure you added both model and recipe
- **Prediction errors:** Make sure you’re predicting on `office_test`

Cross-validation:

- **Folds not created:** Make sure you used `office_train`

- **fit_resamples error:** Make sure you're fitting the workflow, not the fit

If you're stuck:

- Review the walkthrough video
 - Check tidymodels documentation: `?recipe`, `?workflow`, `?vfold_cv`
 - Make sure you completed all previous exercises
 - Ask on the discussion board
-

Key Concepts Review

Feature engineering:

- Creating new variables from existing data
- Aggregating text data (counting lines, detecting keywords)
- Creating indicator variables

Tidymodels workflow:

1. Split data (training/testing)
2. Specify model
3. Create recipe (preprocessing)
4. Build workflow (model + recipe)
5. Fit model
6. Make predictions
7. Evaluate performance

Model evaluation:

- **RMSE:** Root Mean Squared Error (lower is better)
- **R²:** Proportion of variance explained (higher is better)
- **Cross-validation:** More reliable than single train/test split

Why cross-validation?

- Uses all data for training and testing
 - Provides multiple estimates of performance
 - Reduces dependence on single train/test split
 - More reliable estimate of how model will perform on new data
-

Quick Reference

Feature engineering:

```
# Count by group
data %>%
  group_by(group_var) %>%
  summarise(count = n())

# Detect patterns
sum(str_detect(text, "pattern"))

# Create indicators
mutate(indicator = if_else(condition, 1, 0))
```

Tidymodels workflow:


```

# Split
split <- initial_split(data, prop = 0.75)
train <- training(split)
test <- testing(split)

# Model
mod <- linear_reg() %>% set_engine("lm")

# Recipe
rec <- recipe(y ~ ., data = train) %>%
  step_*()

# Workflow
wflow <- workflow() %>%
  add_model(mod) %>%
  add_recipe(rec)

# Fit
fit <- wflow %>% fit(data = train)

# Predict
predictions <- predict(fit, test)

# Cross-validation
folds <- vfold_cv(train, v = 5)
fit_rs <- wflow %>% fit_resamples(folds)
collect_metrics(fit_rs)

```